Session 4 instructor notes — Azure Compute

Objectives: 2.2.1, 2.2.2, 2.2.3, 2.2.4, with a second pass on 1.1.7 **Also practiced:** 3.3.5 (export a template, do not teach ARM yet) **Slides:** session-04-slides.md **Lab:** ../labs/session-04-lab.md **Quiz:** ../quizzes/session-04-quiz.md **Warm-up:** Session 4 set in warm-ups.md, weighted to shared responsibility and IaaS/PaaS **Domain minutes:** Domain 1 — 10, Domain 2 — 95, Domain 3 — 15

Run of show

Min	Block	Leave it when
0–10	Warm-up	OS-patching responsibility and IaaS vs PaaS are back in the room
10–35	Compute types	VM, container, function each have one sentence, and the three container services are not mashed into "Kubernetes"
35–60	VM options and what a VM requires	Scale set vs availability set is distinct; they can name network and disks as required, not only size
60–90	Lab	Both create flows compared. Nothing left running unless you meant it to
90–105	App hosting choice	App Service vs container vs VM is a decision, not a product list
105–120	Quiz	Includes two serverless items

The plan puts hosting options after the lab (Teach C, 15 minutes). Keep that order: the wizard comparison makes the choice slide obvious.

Warm-up emphasis

Prompt 4 (who patches the OS on IaaS) and prompt 5 (one scenario each for IaaS and PaaS). If Session 3's checkpoint showed a shared-responsibility miss, spend the spare minute there.

Teach A

Virtual machine: full OS, you patch it. Container: package, shares the host OS, fast start. Function: event, no server for you to run. Point back at Session 2 serverless in one sentence so 1.1.7 is a second pass, not a new religion.

Then teach the kernel with the house and the apartment. Say it before you name Container Instances, Container Apps, and AKS. Those three are products. This is why a container is not a small virtual machine.

A virtual machine is a standalone house. It has its own plumbing, heating, and foundation. A container is an apartment. It shares the building's plumbing and the rest of the building's infrastructure.

	Virtual machine	Container
Picture	A house with its own foundation	An apartment in a shared building

	Virtual machine	Container
Operating system	Boots its own full guest operating system, Windows or Linux. You patch that system.	Does not boot a guest operating system. It runs on the host computer's existing operating system.
Kernel	Runs its own kernel	Shares the host kernel with every other container on that host
Size and start	Heavy, often gigabytes, and slow to boot because the guest operating system has to load	Light, often megabytes, and starts in seconds

The kernel is the core of an operating system. It is the bridge between software and the physical hardware: CPU, memory, and disks.

Draw two stacks on the board.

- Virtual machine: the app talks to the guest kernel, the guest kernel talks to the hypervisor, and the hypervisor talks to the physical hardware. Those extra layers are why a virtual machine is heavier to start.
- Container: the app talks to the host kernel, and the host kernel gives that container a slice of the hardware. There is no second operating system in the middle, which is why the container starts faster.

If someone asks about security, say one sentence and stop. Sharing a kernel is why containers are light, and it is also why a problem in that kernel can affect every container on the host. A virtual machine's guest kernel is a separate boundary. The exam does not ask you to configure either case. It asks you to know that the container shares the host operating system and the virtual machine does not.

Container services, say the job not the logo:

- Container Instances — "run this container" without a cluster.
- Container Apps — scale and revise containers without you operating Kubernetes.
- AKS — managed Kubernetes when the team needs the orchestrator.

If you are unsure of a current feature boundary between Container Apps and AKS, do not invent one. The exam contrast at this level is simple-container vs managed-Kubernetes. Send detail questions to the module.

Teach B

Scale set: many identical VMs, load balanced, can autoscale. That is scale out as a product.

Availability set: update domains and fault domains so maintenance or a hardware fault does not hit every VM at once. It is an older single-datacenter spreading tool. It is not availability zones. Say "zones are separate datacenters in the region; an availability set is domains inside the datacenter deployment." If you blur these, Session 3's zone work unravels.

Azure Virtual Desktop: desktops and apps delivered from Azure. Not App Service. The confusion is the word "virtual."

VM resources (2.2.3): size, disks, virtual network, subnet, NIC, and usually a public IP if it faces the internet. Open a size name such as `D2s_v5` only long enough to show that the name encodes family and size. Do not teach the price list.

Lab

Session 4 lab. Default is stop at Review + create and discard. Deploying a VM into a class of new accounts is how you get a surprise bill. The Functions guided project is homework-length; in class, do only the slice you rehearsed, or assign the project and spend the half hour on the two wizards.

Template download is a glimpse of 3.3.5. Say "this JSON is the deployment description Azure Resource Manager can run again." Do not explain parameters today.

Teach C

Choice slide, fast, using what they just saw in the two wizards. The VM wizard asked for an OS, a size, and a network. The App Service wizard asked for a runtime and a plan. That difference is IaaS vs PaaS from Session 2.

If you are short on time

Drop Container Apps detail before you drop the scale-set vs availability-set contrast. Drop the guided project before you drop the wizard comparison. Keep the quiz.

Sources

Compute module · VMs · Scale sets · Availability sets · Azure Virtual Desktop · App Service